

Python Functions — Complete Notes (1/7)

1. What Is a Function?

A function is a reusable block of code designed to perform a specific task.

Functions reduce repetition, improve readability, make programs modular, and make testing easier.

2. Creating a Function ☆

```
def function_name():  
    # statements  
    print('Hello')  
  
function_name()
```

3. Function Definition vs Call

Defining a function stores its instructions. Calling it executes those instructions.

4. Parameters & Arguments ☆

```
def greet(name):  
    print('Hello', name)  
  
greet('Jahanur')
```

5. Multiple Parameters

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

6. Why Functions Matter

- Code reuse
- Less duplication
- Break large problems into smaller parts
- Readable and maintainable code
- Useful for DSA and real projects

Python Functions — Complete Notes (2/7)

7. return ☆

return sends a value back to the caller and immediately ends the function.

```
def add(a,b):  
    return a+b  
  
result=add(10,20)  
print(result)
```

8. print() vs return ☆

- *print() displays a value.*
- *return gives a value back to the caller.*
- *Returned values can be stored, compared, reused or passed to another function.*

9. Multiple Return Values

```
def calc(a,b):  
    return a+b, a-b  
  
x,y=calc(10,3)
```

10. None

A function without an explicit return returns None.

```
def hello():  
    print('Hi')  
  
x=hello()  
print(x) # None
```

11. Local Variables

```
def test():  
    x=10  
    print(x)  
  
test()  
print(x) # NameError
```

12. Global Variables

```
x=10  
def show():  
    print(x)  
show()
```

Python Functions — Complete Notes (3/7)

13. Positional Arguments

```
def student(name, age):  
    print(name, age)  
  
student('Jahanur', 22)
```

14. Keyword Arguments ☆

```
def student(name, age):  
    print(name, age)  
  
student(age=22, name='Jahanur')
```

15. Default Arguments ☆

```
def greet(name='User'):  
    print('Hello', name)  
  
greet()  
greet('Jahanur')
```

16. *args ☆

**args collects any number of positional arguments into a tuple.*

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(10, 20, 30))
```

17. **kwargs ☆

***kwargs collects keyword arguments into a dictionary.*

```
def info(**data):  
    print(data)  
  
info(name='Jahanur', age=22)
```

18. Combining args and kwargs

```
def show(*args, **kwargs):  
    print(args)  
    print(kwargs)  
  
show(1, 2, name='Jahanur')
```

Python Functions — Complete Notes (4/7)

19. Argument Unpacking ☆

```
def add(a,b,c):  
    return a+b+c  
  
nums=[10,20,30]  
print(add(*nums))
```

20. Dictionary Unpacking

```
def student(name,age):  
    print(name,age)  
  
data={'name':'Jahanur','age':22}  
student(**data)
```

21. Positional-Only Parameters

```
def add(a,b,/):  
    return a+b  
  
add(2,3)
```

22. Keyword-Only Parameters

```
def greet(*,name):  
    print(name)  
  
greet(name='Jahanur')
```

23. Docstrings ☆

```
def add(a,b):  
    '''Return the sum of a and b.'''  
    return a+b  
  
print(add.__doc__)
```

24. Type Hints

```
def add(a:int,b:int)->int:  
    return a+b
```

25. Good Function Naming

- Use clear descriptive names.
- Use snake_case.
- Prefer names that describe the action: calculate_total(), find_max().

Python Functions — Complete Notes (5/7)

26. Lambda Function ☆

A lambda is a small anonymous function containing one expression.

```
square=lambda x:x*x
print(square(5))
```

27. Lambda with sorted()

```
students=[('A',80),('B',95),('C',70)]
students.sort(key=lambda x:x[1])
```

28. map() and filter()

```
nums=[1,2,3,4,5]
squares=list(map(lambda x:x*x,nums))
evens=list(filter(lambda x:x%2==0,nums))
```

29. Recursion ☆

Recursion is when a function calls itself. Every recursive solution needs a base case.

```
def factorial(n):
    if n==0:
        return 1
    return n*factorial(n-1)
```

30. Base Case vs Recursive Case

- Base case stops recursion.
- Recursive case reduces the problem toward the base case.
- Without a valid base case, recursion can raise RecursionError.

31. LEGB Scope ☆

Python searches names in this order: Local → Enclosing → Global → Built-in.

32. global / nonlocal

global lets a function rebind a global variable. nonlocal lets an inner function rebind a variable from an enclosing function.

Python Functions — Complete Notes (6/7)

33. Functions Are First-Class Objects ☆

- Store a function in a variable.
- Pass a function as an argument.
- Return a function from another function.
- Store functions in collections.

```
def greet():  
    return 'Hello'  
  
f=greet  
print(f())
```

34. Higher-Order Functions

```
def apply(func,value):  
    return func(value)  
  
def double(x): return x*2  
print(apply(double,5))
```

35. Function Returning Function

```
def multiplier(n):  
    def multiply(x):  
        return x*n  
    return multiply  
  
double=multiplier(2)  
print(double(5))
```

36. Closure

A closure remembers values from its enclosing scope even after the outer function has returned.

37. Decorator Concept ☆

A decorator modifies or extends another function's behavior without changing its main code.

```
def decorator(func):  
    def wrapper():  
        print('Before')  
        func()  
        print('After')  
    return wrapper
```

38. Clean Function Design

- Keep one clear responsibility.
- Prefer parameters over hard-coded values.
- Return results when callers need them.
- Avoid unnecessary global state.
- Keep functions readable and testable.

Python Functions — Complete Notes (7/7)

39. Functions + DSA ☆

Functions are especially useful in DSA because each algorithm can be written, tested and reused independently.

```
def find_max(numbers):
    largest=numbers[0]
    for x in numbers:
        if x>largest: largest=x
    return largest
```

40. Frequency Map Function

```
def frequency(numbers):
    freq={}
    for x in numbers:
        freq[x]=freq.get(x,0)+1
    return freq
```

41. Two Sum Function

```
def two_sum(nums, target):
    seen={}
    for i,x in enumerate(nums):
        need=target-x
        if need in seen: return [seen[need],i]
        seen[x]=i
    return []
```

42. Common Mistakes ❓

- Forgetting to call the function.
- Forgetting return when a result is needed.
- Using print() instead of returning a value.
- Unnecessary global variables.
- Mutable default arguments such as x=[].
- Wrong indentation.
- Code after an unconditional return.

43. Practice — Beginner

- Write add(a,b).
- Write is_even(n).
- Find maximum without max().
- Count vowels in a string.
- Reverse a string with a function.

44. Practice — Intermediate

- Remove duplicates with a function.
- Find second largest unique value.
- Return a frequency dictionary.
- Check palindrome.
- Find common elements of two lists.

45. Practice — DSA

- Two Sum
- Binary Search
- Maximum Subarray
- Move Zeroes
- First non-repeating character

46. Final Revision ☆

- def → define function
- call → execute function